Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Johan Chamba
Asignatura	Teoría de la programación
Ciclo	1 A
Unidad	1
Resultado de aprendizaje de la unidad	Identifica los conceptos fundamentales de la teoría de la programación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Práctica Nro.	005
Título de la Práctica	Primer acercamiento a la construcción de algoritmos con estructuras secuenciales Fase 2 - Pseudocódigo
Nombre del Docente	José Oswaldo Guamán Quinche
Fecha	Jueves 15 de octubre del 2026 al Viernes 16 de octubre del 2026
Horario	Jueves, 12h30 – 13h30 Viernes, 09h30 - 11h30
Lugar	Aula física asignada al paralelo
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Familiarizar al estudiante en la creación de algoritmos en diagramas de flujo y pseudocódigo aplicando estructuras secuenciales básicas.

3. Materiales y reactivos

- IDE de programación: Visual Studio Code u otro entorno compatible.
- Lenguaje de programación: Según los contenidos de la unidad.

4. Equipos y herramientas

- Computador personal con sistema operativo Windows, Linux o macOS.
- Material de apoyo en el Entorno Virtual de Aprendizaje (EVA).
- Editores de texto (Word, Google Docs u otros) para la elaboración del informe técnico en formato PDF. FEIRNNR - Carrera de Computación
- Conexión a internet estable para acceder a recursos digitales y software en línea.

5. Procedimiento y metodología

- Se siguió los siguientes pasos para la práctica:
- Primero analice el problema y las fórmulas de la guía
- Se organizo los datos de entrada, el procedimiento necesario (se tuvo que despejar las fórmulas) y los datos de salida
- Se realizo el diagrama de flujo en draw.io
- Se realizo el pseudocódigo en Word
- En base al pseudocódigo y los datos se realizó el código en C con VS Code
- Se verifico los resultados con una prueba de escritorio
- Al terminar con todo lo necesario se empezó con la elaboración del Reporte Técnico en PDF

6. Resultados

Análisis del problema

1. Datos de entrada:

Son los datos que el usuario debe ingresar para realizar el cálculo.

- C1: Nota del primer certamen (real).
- C2: Nota del segundo certamen (real).
- NL: Nota de laboratorio (real).

2. Proceso:

Son las operaciones que realiza el programa para obtener el resultado.

1. Solicitar las notas del primer certamen, segundo certamen y laboratorio.
2. Validar que las notas ingresadas estén entre 0 y 100.
3. Calcular la nota mínima necesaria en el tercer certamen (C3), mediante la fórmula:

$$C3 = \frac{3 \cdot (NF - 0.3 \cdot NL)}{0.7} - C1 - C2$$

4. Evaluar el resultado obtenido:

- Si C3 <= 0, el estudiante ya puede aprobar incluso obteniendo 0 en el tercer certamen.
- Si C3 > 100, el estudiante no puede aprobar ni obteniendo 100 puntos.
- Si C3 > 0 y C3 <= 100, mostrar la nota mínima que necesita en el tercer certamen.

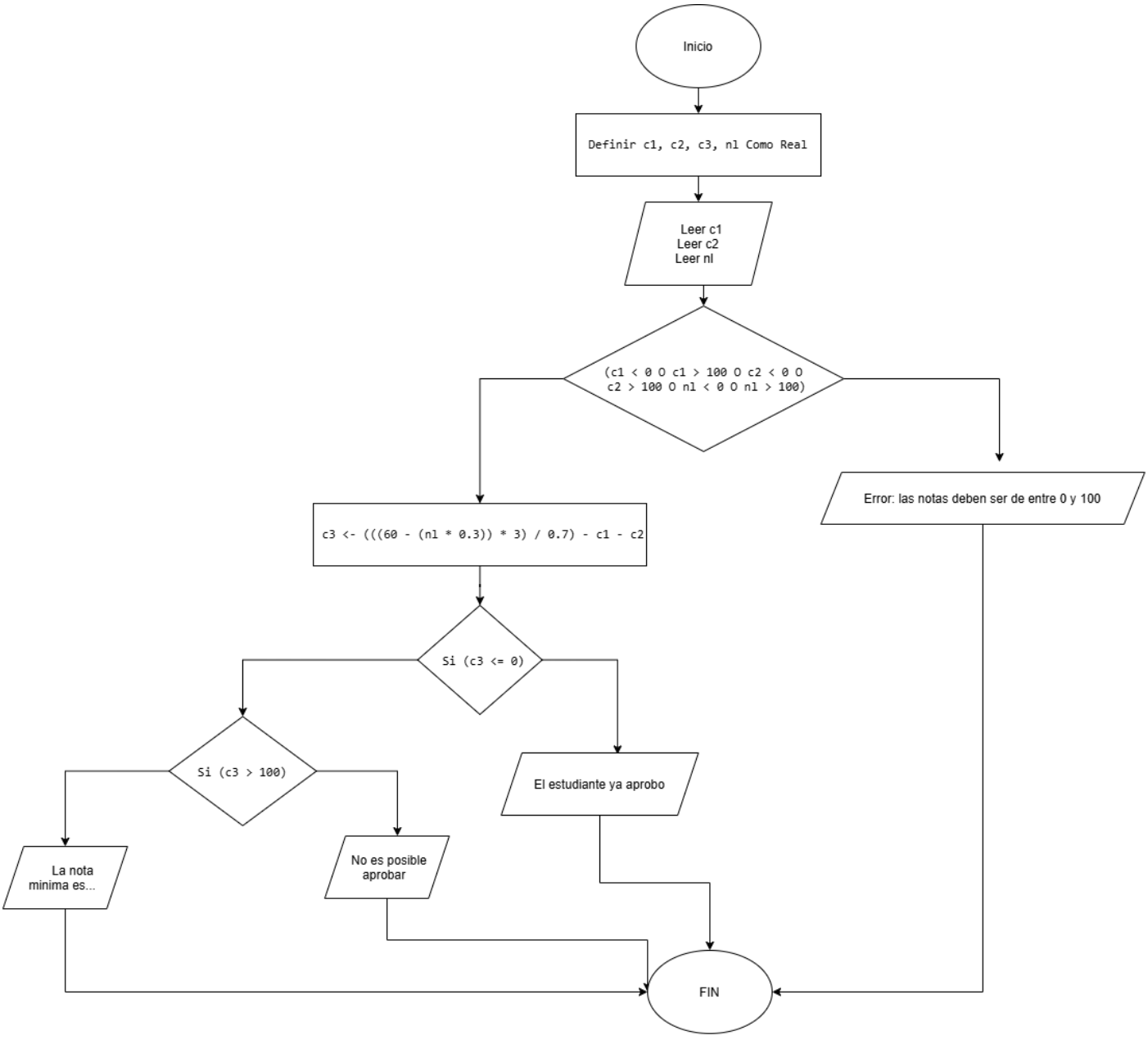
3. Resultados esperados:

Son los resultados que el programa mostrará al finalizar el proceso.

- Mostrar la nota mínima necesaria en el tercer certamen (C3).
- Indicar si el estudiante ya tiene suficientes puntos para aprobar.
- Indicar si no es posible aprobar la asignatura.
- Mostrar un mensaje de error cuando las notas ingresadas estén fuera del rango permitido (0 a 100).

Objetivo: Determinar la nota mínima que necesita obtener un estudiante en el tercer certamen para alcanzar una calificación final de 60/100 y aprobar la asignatura.

Diagrama de flujo



Pseudocódigo

```
Algoritmo Calcular Nota

    Definir c1, c2, c3, nl Como Real

    Escribir "Ingrese la nota del primer certamen:"
    Leer c1

    Escribir "Ingrese la nota del segundo certamen:"
    Leer c2

    Escribir "Ingrese la nota de laboratorio:"
    Leer nl

    Si (c1 < 0 O c1 > 100 O c2 < 0 O c2 > 100 O nl < 0 O nl > 100) Entonces

        Escribir "Error las notas deben estar entre 0 y 100."

    SiNo

        c3 <- (((60 - (nl * 0.3)) * 3) / 0.7) - c1 - c2

        Si (c3 <= 0) Entonces

            Escribir "El estudiante ya aprueba con sus notas actuales."


        SiNo

            Si (c3 > 100) Entonces

                Escribir "No es posible aprobar ni obteniendo 100 en C3."

            SiNo

                Escribir "La nota minima necesaria en C3 es: ", c3

            FinSi

        FinSi

    FinSi

FinAlgoritmo
```

Implementación en C

ape5.c > ...

```c
#include <stdio.h>

int main() {

    float c1;
    float c2;
    float c3;
    float nl;

     printf("Ingrese la nota del primer certamen: ");
    scanf("%f", &c1);

    printf("Ingrese la nota del segundo certamen: ");
    scanf("%f", &c2);

    printf("Ingrese la nota de laboratorio: ");
    scanf("%f", &nl);

     if (c1 < 0 || c1 > 100 || c2 < 0 || c2 > 100 || nl < 0 || nl > 100){

        printf("Error las notas deben estar entre 0 y 100.\n");

     }
     else{

        c3 = (((60 - (nl * 0.3)) * 3) / 0.7) - c1 - c2;

     }
       if (c3 <= 0) {

          printf("El estudiante ya aprueba con sus notas actuales.\n");


       } else if (c3 > 100) {

          printf("No es posible aprobar ni obteniendo 100 en C3.\n");

       } else {

          printf("La nota minima necesaria en C3 es: %.2f\n", c3);

       }
    }
```

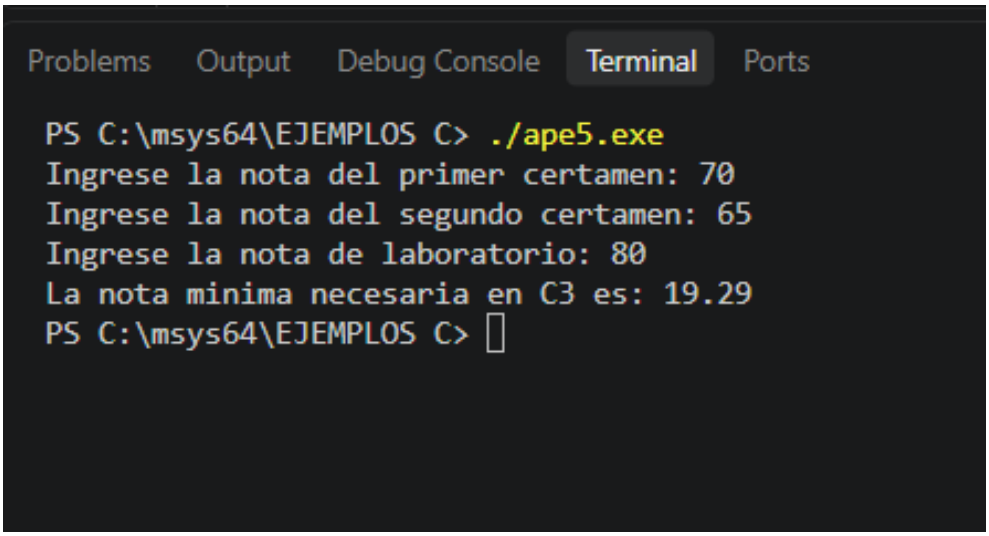
Prueba de escritorio

Se utilizaron los siguientes datos de ejemplo para comprobar los cálculos del algoritmo.

Variable	Valor
C1	70
C2	65
NL	80

Paso	c1 < 0 || c1 > 100 || c2 < 0 || c2 > 100 || 0 < 0 || nl > 100	c3 = (((60 - (nl * 0.3)) * 3) / 0.7) - c1 - c2	C3 <= 0	C3 > 100	Resultados
1	SI	c3 = (((60 - (80 * 0.3)) * 3) / 0.7) - 70 - 65 =19.29	NO	NO	El estudiante necesita obtener 19.29 puntos en el tercer certamen para aprobar.

Resultado: El estudiante necesita obtener 19.29 puntos en el tercer certamen para aprobar.



7. Preguntas de Control

1. ¿Qué es un programa con estructuras secuenciales?

Es un programa que realiza sus instrucciones de manera ordenada, es decir, ejecuta una operación después de otra, siguiendo los pasos establecidos desde el inicio hasta el final. Este tipo de estructura permite que el programa sea más fácil de entender y desarrollar, ya que cada instrucción tiene un orden específico.

2. ¿Cómo se traslada un algoritmo en pseudocódigo o diagrama de flujo a un lenguaje de programación?

Para pasar un algoritmo a un lenguaje de programación, primero debemos entender qué queremos resolver y cuáles son los pasos que debemos seguir. Después, convertimos cada instrucción del pseudocódigo o diagrama de flujo en código, utilizando las herramientas del lenguaje que estamos trabajando, en este caso C. Por ejemplo, utilizamos scanf() para ingresar datos y printf() para mostrar resultados. Finalmente, ejecutamos el programa para comprobar que todo funcione correctamente.

3. ¿Por qué es importante validar el programa mediante casos de prueba?

Es importante porque nos permite comprobar si el programa está funcionando como esperamos. Al realizar diferentes pruebas, podemos encontrar errores, verificar que los cálculos sean correctos y observar cómo responde el programa cuando ingresamos distintos datos. De esta manera, podemos corregir los problemas antes de dar por terminado nuestro trabajo.

4. Conclusiones

- Mediante el desarrollo de esta práctica se logró comprender cómo funcionan las estructuras de programación en C, utilizando variables, operaciones matemáticas y condiciones para resolver un problema de manera ordenada.
- Se aprendió a trasladar un algoritmo del pseudocódigo al lenguaje C, identificando los datos de entrada, los procesos que se deben realizar y los resultados que se esperan obtener.
- La realización de las pruebas de escritorio permitió comprobar que los cálculos fueran correctos en diferentes situaciones, además de identificar posibles errores cuando se ingresan notas fuera del rango establecido.
- La práctica se realizó con éxito, permitiendo fortalecer los conocimientos básicos de programación y comprender la importancia de analizar el problema antes de comenzar a escribir el código.

5. Recomendaciones

1. Se recomienda recibir más clases prácticas sobre el uso del lenguaje de programación C, para comprender mejor cómo escribir y ejecutar los algoritmos.

2. Es importante tener paciencia al momento de realizar los ejercicios, ya que aprender a programar requiere tiempo, práctica y dedicación.

3. Se recomienda verificar cada paso del algoritmo, revisar los datos de entrada, el proceso y los resultados de salida para evitar errores.

4. También es importante revisar el código con atención y realizar pruebas para comprobar que funcione correctamente antes de finalizar la actividad.

5. Finalmente, se recomienda seguir practicando los conocimientos aprendidos y pedir ayuda al docente cuando existan dudas, con el fin de mejorar poco a poco las habilidades de programación.

.

10. Bibliografía / Referencias

Y. Gallardo, “Curso de C” [Video en línea]. YouTube, 9-nov-2024. [En línea]. Disponible en: https://www.youtube.com/watch?v=VEw1jTYWydw

11. Anexos

Querido Ingeniero tuve problemas para subir mi archivo a Git hub espero que me disculpe y me comprometo a que el próximo trabajo lo que tenga como usted quiera, gracias.